ER Diagram, Indexing, Database Normalization

Day 2

Topic Covered :

ER Diagram

1. Entity
2. Attribute
3. Relationship
4. Cardinality
5. Participation
6. Primary Key
7. Foreign Key
8. Designing ER Diagram

Indexing
9. What is Index
10. Why Index
11. Internal Working
12. B Tree
13. Clustered Index
14. Non Clustered Index
15. Composite Index
16. Unique Index
17. Covering Index
18. Performance Analysis

Database Normalization
19. Data Redundancy
20. Functional Dependency
21. 1NF
22. 2NF
23. 3NF
24. BCNF
25. Denormalization

Mini Overview

- We formalize yesterday's rough ER sketch into a proper, industry-standard ER Diagram — with correct notation for entities, attributes, relationships, cardinality (1:1, 1:N, M:N), and participation constraints.
- We go deep into Indexing — not just "what speeds up queries," but the actual B-Tree internals that make indexes work, and the different index types (clustered, non-clustered, composite, unique, covering) with performance comparisons.
- We cover Normalization from first principles — functional dependency, redundancy, and each normal form (1NF → BCNF) — so our schema is mathematically sound, not just "it works."

- We close with Denormalization — knowing when to deliberately break normalization rules for performance, a real-world trade-off senior engineers must judge.

Expected Outcome

- You'll be able to draw a correct, fully-notated ER diagram for any given business scenario — not just the Health Clinic.
- You'll understand *why* an index makes a query fast at the internal B-Tree level, not just "add an index and it's faster."
- You'll be able to take an unnormalized table and systematically normalize it to 3NF/BCNF, explaining each step's justification.
- You'll finalize the **complete ER Diagram** for the Health Clinic Application, normalize its schema, and create proper indexes — directly usable in Day 3 (Joins/Procedures/Triggers) and Day 4 (JDBC final project).

Part 1: ER Diagram Fundamentals — Entity, Attribute, Relationship, Cardinality, Participation, Keys

---

1. Entity

Definition:
An entity is any real-world object or concept that has independent existence and can be distinctly identified — it becomes a table in the final database.

Types of Entities:

- **Strong Entity**: Has its own primary key, exists independently (e.g., `Patient`, `Doctor`)
- **Weak Entity**: Cannot be uniquely identified by its own attributes alone; depends on a strong entity (e.g., `Dependent` of a patient(prescription, bills, medicines) — identified only in combination with the patient's key)

Notation:

ASCII Diagram: Entity Notation

```
Strong Entity:      Weak Entity:
+------------+      +================+
| PATIENT    |      ‖ DEPENDENT      ‖  (double-lined rectangle)
+------------+      +================+
```

Real-world Example (Health Clinic):
`Patient`, `Doctor`, `Appointment`, `Billing`, `Specialization` — each becomes a table.

Interview Question:

- Q: What's the difference between a strong entity and a weak entity?
- A: A strong entity has its own primary key and exists independently; a weak entity depends on a strong (owner) entity for identification and typically uses a partial key combined with the owner's primary key.

---

2. Attribute

Definition:
An attribute is a property or characteristic of an entity (becomes a column in the table).

Types of Attributes:

Type	Description	Example
Simple	Cannot be divided further	`age`
Composite	Can be broken into sub-parts	`name` → `first_name` + `last_name`
Single-valued	Only one value per entity	`date_of_birth`
Multi-valued	Can have multiple values	`phone_numbers` (a patient may have 2 numbers)
Derived	Calculated from another attribute	`age` derived from `date_of_birth`
Key Attribute	Uniquely identifies the entity	`patient_id`

Notation:

ASCII Diagram: Attribute Notation

```
   ( age )        <- oval = simple attribute
     |
  [ PATIENT ]
     |
   ( name )
    /   \
(first)  (last)     <- composite attribute
     |
  ((phone_numbers))  <- double oval = multi-valued
     |
  ⟨ age ⟩ - - -      <- dashed oval = derived attribute
```

Best Practice:
Avoid storing derived attributes (like `age`) directly in the table — calculate them on the fly from `date_of_birth` at query time. Storing derived data risks it becoming stale/inconsistent.

Common Mistake:
Storing `age` as a static column — it becomes wrong the moment time passes without an update job.

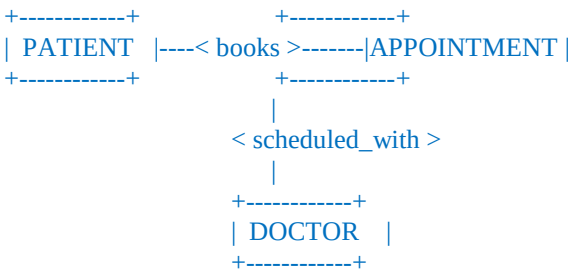
---

3. Relationship

Definition:
A relationship describes how two or more entities are associated with each other. It becomes a foreign key (or a junction table for many-to-many) in the final schema.

Notation:

ASCII Diagram: Relationship Notation



Diamond shapes (Chen notation) represent relationships: `books`, `scheduled_with`, `generates`, etc.

Real-world Example:

- `PATIENT` **books** `APPOINTMENT`
- `DOCTOR` **is scheduled for** `APPOINTMENT`
- `APPOINTMENT` **generates** `BILLING`

Interview Question:

- Q: How does a relationship in an ER diagram translate into an actual database structure?
- A: A 1:1 or 1:N relationship typically becomes a foreign key in the "many" side table; a Many:Many relationship requires a separate junction/bridge table containing foreign keys from both entities.

---

4. Cardinality

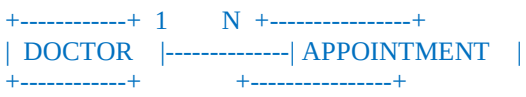
Definition:
Cardinality defines the numerical relationship between rows of two entities — how many instances of one entity relate to how many instances of another.

Types:

a) One-to-One (1:1)

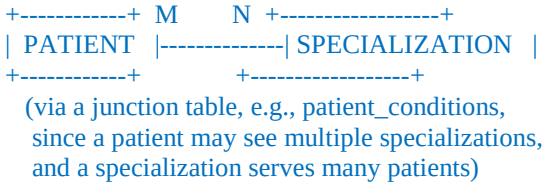
Example: One doctor has exactly one detailed profile record.

b) One-to-Many (1:N)



Example: One doctor can have many appointments, but each appointment belongs to only one doctor.

c) Many-to-Many (M:N)



How M:N is Implemented in RDBMS:
A Many-to-Many relationship cannot be directly implemented with a simple foreign key — it requires a **junction/bridge table**:

sql

```
CREATE TABLE patient_specializations (
    patient_id INT,
    specialization_id INT,
    PRIMARY KEY (patient_id, specialization_id),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (specialization_id) REFERENCES specializations(specialization_id)
);
```

Interview Question:

- Q: Why can't a Many-to-Many relationship be implemented with a simple foreign key?
- A: A foreign key can only reference one row on the "one" side. In M:N, both sides can have multiple related rows, so a junction table is required to break the M:N relationship into two 1:N relationships.

Common Mistake:
Trying to store multiple foreign key IDs in a single column (e.g., `doctor_ids = "101,102,103"`) instead of creating a proper junction table — this violates 1NF (covered later today) and makes querying/joining nearly impossible efficiently.

---

5. Participation

Definition:
Participation defines whether *every* instance of an entity must participate in a relationship (Total/Mandatory) or if it's optional (Partial).

Types:

a) Total Participation (Mandatory) — double line in Chen notation

Example: Every appointment **must** result in a billing record (mandatory — total participation from Appointment's side).

b) Partial Participation (Optional) — single line

Example: A doctor may exist in the system without having any appointments yet (optional — partial participation).

Real-world Example (Health Clinic):

- `Patient` → `Appointment`: **Partial** (a newly registered patient may not have booked any appointment yet)
- `Appointment` → `Billing`: **Total** (every completed appointment must generate a bill)

Interview Question:

- Q: What's the difference between total and partial participation, and why does it matter for schema design?
- A: Total participation means every entity instance *must* have a corresponding related row (often implemented via `NOT NULL` foreign keys or application-level enforcement); partial participation means the relationship is optional (foreign key can be `NULL`). It matters because it determines whether your foreign key column allows `NULL` and whether you need additional application/trigger-level enforcement.

---

6. Primary Key

Definition:
A Primary Key is a column (or combination of columns) that uniquely identifies each row in a table. It cannot be `NULL` and must be unique.

Rules:

- Unique across all rows
- Cannot be `NULL`
- Only one primary key per table (but it can be composite — multiple columns)

Syntax:

sql

```
CREATE TABLE doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    ...
);

-- Composite Primary Key example
CREATE TABLE patient_specializations (
    patient_id INT,
    specialization_id INT,
    PRIMARY KEY (patient_id, specialization_id)
);
```

Best Practice:
Prefer a **surrogate key** (auto-increment `INT` like `doctor_id`) over a **natural key** (like `phone_number` or `email`) as the primary key — natural keys can change over time (a person changes their phone number), which would break all foreign key references.

Interview Question:

- Q: Why prefer a surrogate key over a natural key as primary key?
- A: Natural keys (email, phone) can change, are sometimes not truly unique, and may contain sensitive data. Surrogate keys (auto-increment integers) are stable, small, and never need to change, keeping foreign key relationships intact permanently.

---

7. Foreign Key

Definition:
A Foreign Key is a column in one table that references the Primary Key of another table, establishing and enforcing a relationship between them.

Syntax:

sql

```sql
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATETIME,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

Referential Integrity Actions:

sql

```sql
FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
    ON DELETE CASCADE     -- if patient deleted, delete their appointments too
    ON UPDATE CASCADE     -- if patient_id changes, update references automatically

-- Other options: ON DELETE RESTRICT (default-like, prevents deletion if referenced)
--                ON DELETE SET NULL (requires FK column to be nullable)
```

Why it is needed:
Prevents **orphaned records** — e.g., an appointment referencing a `patient_id` that doesn't exist. This is what enforces the "relational" part of RDBMS.

Best Practice:
Think carefully before using `ON DELETE CASCADE` in a healthcare system — deleting a patient probably shouldn't silently delete their entire appointment/billing history (audit/legal requirements). Often `ON DELETE RESTRICT` or a "soft delete" (an `is_active` flag) is safer than a hard cascading delete.

Interview Question:

- Q: What's the difference between `ON DELETE CASCADE` and `ON DELETE RESTRICT`?
- A: `CASCADE` automatically deletes/updates dependent rows when the referenced row is deleted/updated. `RESTRICT` (MySQL's default behavior) prevents the deletion of a referenced row if dependent rows still exist, throwing a foreign key constraint error instead.

Common Mistake:
Forgetting to index foreign key columns — MySQL's InnoDB automatically creates an index on FK columns, but always verify this, since a missing index on a heavily-joined FK column causes serious performance issues (connects directly to today's Indexing topic).

---

8. Designing an ER Diagram — Step-by-Step Process

Step-by-step methodology:

1. **Identify Entities** — nouns in the requirements (Patient, Doctor, Appointment, Billing, Specialization)
2. **Identify Attributes** for each entity
3. **Identify Relationships** between entities (verbs: "books," "generates," "assigned to")
4. **Determine Cardinality** for each relationship (1:1, 1:N, M:N)
5. **Determine Participation** (total/partial) for each side
6. **Identify Primary Keys** for each entity
7. **Identify Foreign Keys** based on relationships
8. **Resolve Many-to-Many** relationships into junction tables
9. **Draw the final diagram** and validate against real-world business rules

Best Practice:

Always validate your ER diagram against actual business questions: "Can one doctor have multiple specializations?" "Can an appointment happen without a doctor assigned yet (e.g., walk-in queue)?" These questions often reveal cardinality/participation you initially got wrong.

Part 2: Complete ER Diagram — Health Clinic Application

Applying the 9-Step Methodology to Our Health Clinic

Step 1 — Entities Identified:

`Patient`, `Doctor`, `Specialization`, `Appointment`, `Billing`, `VisitHistory`

Step 2 — Attributes:

- Patient: patient_id, first_name, last_name, dob, gender, phone_number, email
- Doctor: doctor_id, first_name, last_name, phone_number, email
- Specialization: specialization_id, name, description
- Appointment: appointment_id, patient_id (FK), doctor_id (FK), appointment_date, status
- Billing: bill_id, appointment_id (FK), amount, payment_status, billing_date
- +VisitHistory: visit_id, appointment_id (FK), diagnosis, prescription, visit_notes

Step 3 & 4 — Relationships & Cardinality:

- Patient **books** Appointment → 1:N (one patient, many appointments)
- Doctor **attends** Appointment → 1:N (one doctor, many appointments)
- Doctor **has** Specialization → M:N (a doctor can have multiple specializations; a specialization applies to many doctors) — **needs junction table**
- Appointment **generates** Billing → 1:1 (each appointment produces exactly one bill)
- Appointment **produces** VisitHistory → 1:1 (each appointment has one visit record)

Step 5 — Participation:

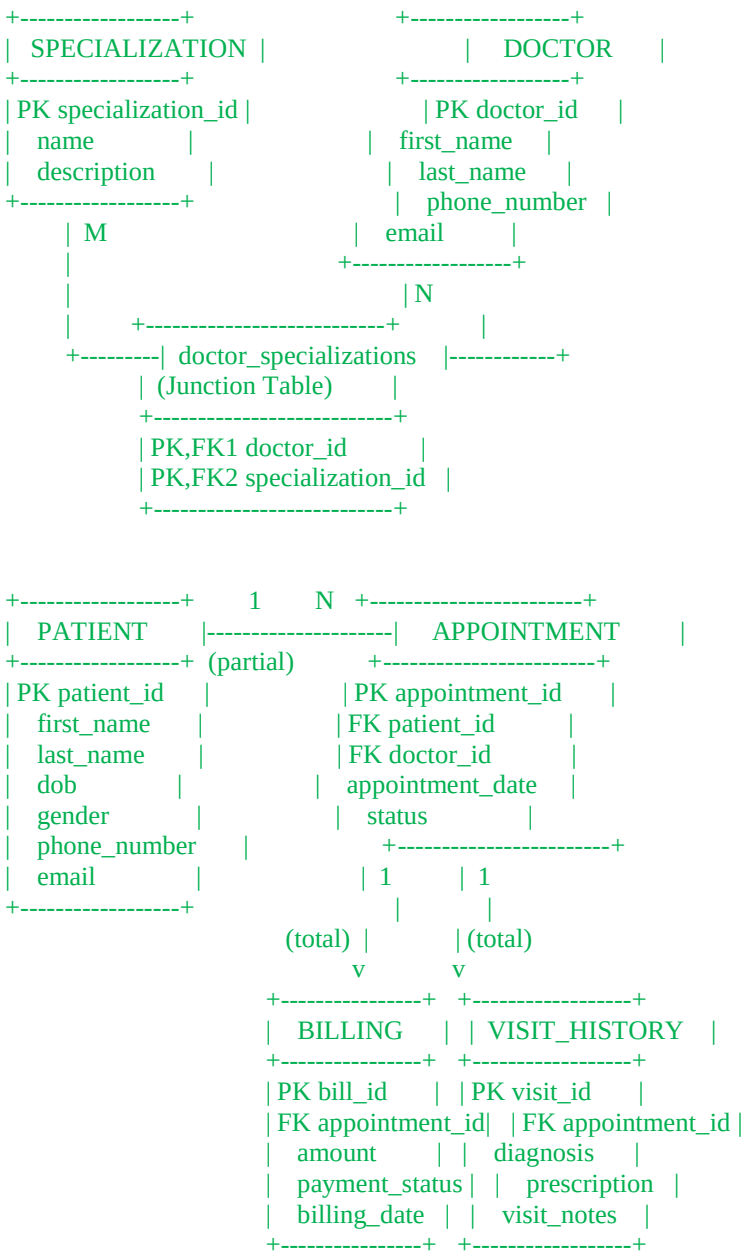
- Patient → Appointment: **Partial** (patient may not have booked yet)
- Doctor → Appointment: **Partial** (doctor may have zero appointments)
- Appointment → Billing: **Total** (every appointment must be billed)
- Appointment → VisitHistory: **Total** (every completed appointment must have visit notes)

Steps 6–8 — Keys & Junction Table Resolution:

- All entities get surrogate `INT AUTO_INCREMENT` primary keys
- Doctor↔Specialization (M:N) resolved via junction table `doctor_specializations`

Complete ER Diagram (ASCII)

ASCII Diagram: Health Clinic — Complete ER Diagram



DOCTOR --- N --- ATTENDS ---- N --- APPOINTMENT
(also linked via appointment.doctor_id, shown above)

Relationship Summary Table:

Relationship	Cardinality	Participation
Patient → Appointment	1:N	Partial (Patient side)
Doctor → Appointment	1:N	Partial (Doctor side)
Doctor ↔ Specialization	M:N	Partial both sides
Appointment → Billing	1:1	Total (Appointment side)
Appointment → VisitHistory	1:1	Total (Appointment side)

Interview Question:

- Q: Why is `doctor_specializations` needed as a separate table instead of adding a `specialization_id` column directly to the `doctors` table?
- A: Because the relationship is Many-to-Many — a doctor can have multiple specializations, and a specialization can apply to multiple doctors. A single foreign key column can only model a 1:N relationship, so a junction table with a composite primary key (`doctor_id`, `specialization_id`) is required to properly model M:N.

Common Mistake:

Modeling `Billing` and `VisitHistory` as attributes directly inside the `Appointment` table instead of separate tables. While tempting for a 1:1 relationship, keeping them separate maintains single responsibility per table, allows independent evolution (e.g., adding more billing fields later), and is more consistent with normalization principles (coming up next).

Part 3: Indexing

---

9. What is an Index?

Definition:
An index is a special lookup data structure that the database engine maintains alongside a table, allowing it to find rows much faster than scanning the entire table row-by-row.

Analogy:
Think of a book's index at the back — instead of reading every page to find "Normalization," you look it up in the index and jump directly to page 47.

---

10. Why Index?

Why it is needed:
Without an index, MySQL must perform a **full table scan** — checking every single row — to find matching data. For a table with millions of rows (e.g., years of appointment history), this is extremely slow.

Real-world Example:

sql

```
-- Without an index on patient_id, this scans ALL rows in appointments
SELECT * FROM appointments WHERE patient_id = 105;
```

With an index on `patient_id`, MySQL jumps almost directly to the matching rows.

Trade-off (important, often missed):
Indexes speed up `SELECT`/`WHERE`/`JOIN` but **slow down** `INSERT`/`UPDATE`/`DELETE`, because the index structure itself must be updated every time data changes. This is why you shouldn't index every column blindly.

---

11. Internal Working of Indexes

Definition:
Internally, MySQL's InnoDB storage engine uses a **B+ Tree** data structure to implement indexes (not a hash table, though `MEMORY` engine tables can use hash indexes).

How it works conceptually:

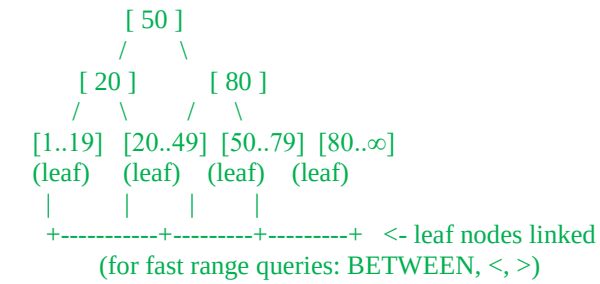
- The tree is sorted by the indexed column's value
- Each "search" starts at the root node and traverses down to a leaf node, following the sorted structure
- This reduces search complexity from O(n) (full scan) to O(log n) (tree traversal)

---

12. B-Tree (B+ Tree specifically in InnoDB)

Definition:
A B+ Tree is a self-balancing tree data structure where all actual data (or pointers to data) is stored in leaf nodes, and internal nodes only store keys for navigation. Leaf nodes are linked together for fast range scans.

ASCII Diagram: B+ Tree Index on patient_id



Why B+ Tree specifically (not plain Binary Tree or Hash):

- **Balanced** — guarantees O(log n) lookup regardless of insert order
- **Range-query friendly** — leaf nodes are linked, so `WHERE age BETWEEN 20 AND 40` can scan sequentially across linked leaves instead of re-traversing the tree per row
- **Disk-friendly** — each node maps closely to a disk page, minimizing disk I/O reads

Interview Question:

- Q: Why does MySQL InnoDB use B+ Tree instead of a Hash Table for indexes?
- A: Hash tables give O(1) lookup for exact-match queries but cannot efficiently support range queries (<, >, `BETWEEN`) or sorting (`ORDER BY`), since hashing destroys order. B+ Trees maintain sorted order and support both fast exact lookups (O(log n)) and efficient range scans via linked leaf nodes.

---

13. Clustered Index

Definition:
A clustered index determines the **physical storage order** of the actual table data — the table data itself is stored in the leaf nodes of this index. Each table can have only **one** clustered index.

In InnoDB:
The **Primary Key is automatically the clustered index**. If no primary key exists, InnoDB picks the first `UNIQUE NOT NULL` column, or internally generates a hidden row ID.

ASCII Diagram: Clustered Index (on patient_id, the PK)

```
            [ patient_id: 50 ]
           /                  \
    [patient_id: 20]        [patient_id: 80]
       |                       |
  Leaf: FULL ROW DATA     Leaf: FULL ROW DATA
  (id=20, name=Sita, ...)  (id=80, name=Ravi, ...)
```

Best Practice:
Choose a primary key that is small, sequential, and rarely changes (like `AUTO_INCREMENT INT`) — since it dictates physical row order, a poorly chosen clustered index (e.g., a random UUID) causes expensive page splits/reorganization on every insert.

---

14. Non-Clustered (Secondary) Index

Definition:
A non-clustered (secondary) index stores the indexed column's value **plus a pointer back to the clustered index key** (in InnoDB, this pointer is the Primary Key value) — not the full row data.

ASCII Diagram: Non-Clustered Index (on phone_number)

```
      [ phone_number: '98765...' ]
                |
        points to -> patient_id: 20
                |
        (Then InnoDB does a SECOND lookup
         using the clustered index to get
         the full row using patient_id=20)
```

Key Difference from Clustered:
A query using a non-clustered index often requires **two lookups**: first in the secondary index to find the primary key, then in the clustered index to fetch the full row (unless it's a "covering index" — see below).

Real-world Example:

```sql
CREATE INDEX idx_phone_number ON patients(phone_number);
```

This creates a secondary index, speeding up `WHERE phone_number = '...'` searches without changing the physical row order (which stays sorted by `patient_id`).

---

15. Composite Index

Definition:
A composite (multi-column) index is built on two or more columns together, useful when queries frequently filter on the same combination of columns.

Syntax:

sql

```
CREATE INDEX idx_doctor_date ON appointments(doctor_id, appointment_date);
```

Critical Rule — Leftmost Prefix:
A composite index on `(doctor_id, appointment_date)` can be used for:

- `WHERE doctor_id = 5` ✓ (uses index)
- `WHERE doctor_id = 5 AND appointment_date = '2026-08-01'` ✓ (uses index fully)
- `WHERE appointment_date = '2026-08-01'` ✗ (does NOT use this index — skips the leftmost column)

ASCII Diagram: Composite Index Leftmost Prefix Rule

Index sorted by: (doctor_id, appointment_date)

doctor_id=1, date=Jan-01
doctor_id=1, date=Jan-05
doctor_id=2, date=Jan-01 <- sorted primarily by doctor_id,
doctor_id=2, date=Jan-03 then by date WITHIN each doctor_id
doctor_id=3, date=Jan-02

Searching by date alone (skipping doctor_id)
cannot use this sorted structure efficiently.

Interview Question:

- Q: If you have a composite index on `(A, B)`, will a query filtering only on `B` use this index?
- A: No — composite indexes follow the "leftmost prefix" rule. The index can only be used if the query includes the leftmost column(s) of the index. A query filtering only on `B` would require a separate index on `B` alone.

---

16. Unique Index

Definition:
A unique index enforces that all values in the indexed column(s) are distinct, while also providing the performance benefits of a regular index.

Syntax:

sql

```
CREATE UNIQUE INDEX idx_email ON patients(email);
-- or directly in table definition:
email VARCHAR(100) UNIQUE
```

Difference from Primary Key:

- A table can have only **one** Primary Key, but **multiple** Unique indexes/constraints
- A Primary Key column cannot be `NULL`; a Unique-indexed column **can** have `NULL` (multiple `NULL`s are allowed, since `NULL ≠ NULL` in SQL logic)

---

17. Covering Index

Definition:
A covering index is one where **all columns requested by a query** are present within the index itself, so MySQL never needs to go back to the clustered index (table data) to fetch additional columns — avoiding the "second lookup" mentioned earlier.

Example:

sql

```
CREATE INDEX idx_covering ON appointments(doctor_id, appointment_date, status);

-- This query is fully "covered" by the index above:
SELECT doctor_id, appointment_date, status
FROM appointments
WHERE doctor_id = 5;
-- MySQL never touches the actual table rows -- huge performance win
```

Best Practice:
For frequently-run reporting queries that select a small, consistent set of columns, design a covering index to eliminate the extra lookup entirely.

Interview Question:

- Q: What makes an index a "covering index" for a given query?
- A: When every column referenced in the `SELECT`, `WHERE`, and `ORDER BY` clauses of a query is included in the index itself, so the database engine can satisfy the entire query from the index alone, without accessing the underlying table data.

---

18. Performance Analysis — Using EXPLAIN

Practical Demonstration:

sql

```
-- Without index
EXPLAIN SELECT * FROM appointments WHERE patient_id = 105;
+----+-------------+--------------+------+---------------+------+---------+------+------+-------------+
| id | select_type | table        | type | possible_keys | key  | key_len | ref  | rows | Extra       |
+----+-------------+--------------+------+---------------+------+---------+------+------+-------------+
| 1  | SIMPLE      | appointments | ALL  | NULL          | NULL | NULL    | NULL | 5000 | Using where |
+----+-------------+--------------+------+---------------+------+---------+------+------+-------------+
```

`type: ALL` = full table scan across 5000 rows — bad.

sql

```
-- After adding an index
CREATE INDEX idx_patient_id ON appointments(patient_id);
EXPLAIN SELECT * FROM appointments WHERE patient_id = 105;
+----+-------------+--------------+------+---------------+----------------+---------+-------+------+-------+
| id | select_type | table        | type | possible_keys | key            | key_len | ref   | rows | Extra |
+----+-------------+--------------+------+---------------+----------------+---------+-------+------+-------+
| 1  | SIMPLE      | appointments | ref  | idx_patient_id | idx_patient_id | 4       | const | 3    | NULL  |
+----+-------------+--------------+------+---------------+----------------+---------+-------+------+-------+
```

`type: ref` = index used, scanning only 3 matching rows instead of 5000 — massive improvement.

Best Practice:
Always index foreign key columns and columns frequently used in `WHERE`, `JOIN`, and `ORDER BY` clauses. But don't over-index — each additional index adds overhead to every `INSERT`/`UPDATE`/`DELETE`.

Common Mistake:
Adding an index on a low-cardinality column (e.g., `gender` with only 'Male'/'Female'/'Other') — the optimizer often ignores such indexes anyway since a full scan can be just as fast when there are only a few distinct values.

Interview Question:

- Q: Why wouldn't you index a column like `gender` with only 3 possible values?
- A: Low-cardinality columns (few distinct values) don't benefit much from indexing — since a large fraction of the table matches any given value, the optimizer often chooses a full table scan anyway, and the index adds unnecessary write overhead without meaningful read benefit.

Part 4: Database Normalization

---

19. Data Redundancy

Definition:
Data redundancy occurs when the same piece of data is unnecessarily duplicated across multiple rows or tables, leading to wasted storage and — more importantly — data inconsistency risk.

Redundancy = duplicate data stored unnecessarily, leading to waste and inconsistency.

Example of a BAD, unnormalized table:

sql

```
-- Unnormalized appointments table (BAD DESIGN — for illustration only)
CREATE TABLE appointments_bad (
    appointment_id INT,
    patient_name VARCHAR(100),
    patient_phone VARCHAR(15),
    doctor_name VARCHAR(100),
    doctor_specialization VARCHAR(100),
    appointment_date DATE
);
```

```
+----------------+--------------+---------------+-------------+-----------------------+------------------+
| appointment_id | patient_name | patient_phone | doctor_name | doctor_specialization  | appointment_date |
+----------------+--------------+---------------+-------------+-----------------------+------------------+
| 1              | Ramesh Kumar | 9876543210    | Dr. Rao     | Cardiology            | 2026-08-01       |
| 2              | Ramesh Kumar | 9876543210    | Dr. Rao     | Cardiology            | 2026-08-10       |
| 3              | Sita Sharma  | 9876543211    | Dr. Iyer    | Pediatrics            | 2026-08-02       |
+----------------+--------------+---------------+-------------+-----------------------+------------------+
```

Problems this causes:

1. **Update Anomaly** — If Ramesh's phone number changes, we must update it in every row it appears — miss one, and data becomes inconsistent.
2. **Insertion Anomaly** — We can't add a new doctor to the system until they have at least one appointment (since doctor data only exists inside the appointments table).
3. **Deletion Anomaly** — If we delete appointment_id 3 (Sita's only appointment), we lose all information about Dr. Iyer and Pediatrics entirely.

This is exactly why Normalization exists — to eliminate these three anomalies by restructuring data into smaller, related tables.

---

20. Functional Dependency

Definition:
A functional dependency exists when one attribute (or set of attributes) uniquely determines another attribute. Written as `A → B` ("A determines B"), meaning for every value of A, there is exactly one corresponding value of B.

Example:

patient_id → patient_name, patient_phone
(Given a patient_id, the name and phone are uniquely determined)

doctor_id → doctor_name, doctor_specialization
(Given a doctor_id, the doctor's name and specialization are uniquely determined)

Types of Functional Dependency:

- **Full Functional Dependency**: An attribute depends on the *entire* composite key, not just part of it.
- **Partial Functional Dependency**: An attribute depends on only *part* of a composite key (this is the problem 2NF fixes).
- **Transitive Dependency**: A non-key attribute depends on another non-key attribute, rather than directly on the primary key (this is the problem 3NF fixes).

Interview Question:

- Q: What is a functional dependency, and why is it foundational to normalization?
- A: A functional dependency `A → B` means A's value uniquely determines B's value. Normalization rules (2NF, 3NF, BCNF) are formally defined in terms of eliminating "bad" functional dependencies (partial and transitive) that cause data redundancy and anomalies.

---

21. First Normal Form (1NF)

Rule: Every column must hold **atomic** (indivisible) values, and each row must be uniquely identifiable. No repeating groups or multi-valued columns.

***Normalization** in databases means **organizing data into well-structured tables to reduce redundancy and improve consistency**. It's a process of breaking down large, messy tables into smaller ones and linking them with relationships (usually foreign keys).*

Violation Example:

```
+----+--------------+------------------------+
| id | patient_name | phone_numbers          |
+----+--------------+------------------------+
| 1  | Ramesh Kumar | 9876543210, 9998887777 |   <- multi-valued, NOT atomic
+----+--------------+------------------------+
```

1NF Fix:

sql

```
-- Separate table for multi-valued phone numbers
CREATE TABLE patient_phones (
    phone_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    phone_number VARCHAR(15),
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id)
);
+----------+------------+---------------+
| phone_id | patient_id | phone_number  |
+----------+------------+---------------+
| 1        | 1          | 9876543210    |
| 2        | 1          | 9998887777    |
+----------+------------+---------------+
```

Interview Question:

- Q: What violates 1NF, and how do you fix it?
- A: Storing multiple values in a single column (e.g., comma-separated phone numbers) violates 1NF. Fix: move the multi-valued attribute into a separate related table with a foreign key back to the original entity.

---

22. Second Normal Form (2NF)

Rule: Must already be in 1NF, **and** every non-key attribute must depend on the **entire** primary key (no partial dependency). This rule only applies meaningfully when you have a **composite primary key**.

Violation Example:

sql

```
-- Composite PK: (appointment_id, doctor_id) -- BAD design
CREATE TABLE appointment_details_bad (
    appointment_id INT,
    doctor_id INT,
    appointment_date DATE,     -- depends only on appointment_id (partial dependency)
    doctor_specialization VARCHAR(100),  -- depends only on doctor_id (partial dependency)
    PRIMARY KEY (appointment_id, doctor_id)
);
```

Here, `appointment_date` depends only on `appointment_id` (not the full composite key), and `doctor_specialization` depends only on `doctor_id` — both are **partial dependencies**, violating 2NF.

2NF Fix — Split into separate tables:

sql

```sql
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_date DATE
);

CREATE TABLE doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    specialization VARCHAR(100)
);

CREATE TABLE appointment_doctor_map (
    appointment_id INT,
    doctor_id INT,
    PRIMARY KEY (appointment_id, doctor_id),
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

Interview Question:

- Q: When does 2NF violation become a concern?
- A: Only when a table has a **composite primary key**. If an attribute depends on only part of that composite key rather than the whole key, it's a partial dependency — violating 2NF — and should be moved to a separate table keyed on just that part.

---

23. Third Normal Form (3NF)

Rule: Must already be in 2NF, **and** no non-key attribute should depend on another non-key attribute (no transitive dependency). Every non-key column must depend **only** on the primary key — directly.

Violation Example:

sql

```sql
-- BAD: doctor_specialization depends on doctor_id, NOT on appointment_id (the PK)
CREATE TABLE appointments_bad (
    appointment_id INT PRIMARY KEY,
    doctor_id INT,
    doctor_name VARCHAR(100),          -- transitively depends on doctor_id
    doctor_specialization VARCHAR(100), -- transitively depends on doctor_id
    appointment_date DATE
);
```

Here: `appointment_id → doctor_id → doctor_name, doctor_specialization`. Since `doctor_name` and `doctor_specialization` depend on `doctor_id` (a non-key attribute), not directly on `appointment_id` (the actual primary key), this is a **transitive dependency** — violating 3NF.

3NF Fix:

sql

```
CREATE TABLE doctors (
   doctor_id INT AUTO_INCREMENT PRIMARY KEY,
   doctor_name VARCHAR(100),
   specialization VARCHAR(100)
);

CREATE TABLE appointments (
   appointment_id INT AUTO_INCREMENT PRIMARY KEY,
   doctor_id INT,
   appointment_date DATE,
   FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id)
);
```

Now `doctor_name` and `specialization` live in `doctors`, referenced via `doctor_id` — no transitive dependency remains.

Interview Question:

- Q: What is a transitive dependency, and give a real-world example from a healthcare schema?
- A: A transitive dependency is when a non-key attribute depends on another non-key attribute rather than directly on the primary key. Example: in an unnormalized `appointments` table, `doctor_specialization` depends on `doctor_id` (not on `appointment_id`, the actual key) — this indirect chain is transitive and should be normalized into a separate `doctors` table.

---

24. Boyce-Codd Normal Form (BCNF)

Rule: A stricter version of 3NF. For every functional dependency `A → B`, `A` must be a **super key** (a candidate key or a superset of one). 3NF has a narrow exception for certain overlapping candidate keys; BCNF removes that exception entirely.

When 3NF ≠ BCNF (classic case):
This happens when a table has **multiple overlapping candidate keys**.

Example:

```
Table: doctor_specialization_room
(Assume: each doctor can practice ONE specialization in a given room,
 and each room is dedicated to only ONE specialization)

+-----------+------------------+--------+
| doctor_id | specialization   | room   |
+-----------+------------------+--------+
| 101       | Cardiology       | R1     |
| 102       | Cardiology       | R1     |
| 103       | Pediatrics       | R2     |
+-----------+------------------+--------+
```

```
Functional Dependencies:
(doctor_id, specialization) → room      [composite key]
room → specialization                   [room alone determines specialization,
                                        but room is NOT a candidate key by itself]
```

`room → specialization` violates BCNF because `room` is not a super key, even though the table technically satisfies 3NF (specialization is a prime attribute since it's part of a candidate key, so the 3NF exception applies).

BCNF Fix — Decompose further:

sql

```
CREATE TABLE room_specialization (
   room VARCHAR(10) PRIMARY KEY,
   specialization VARCHAR(100)
);

CREATE TABLE doctor_room (
   doctor_id INT,
   room VARCHAR(10),
   PRIMARY KEY (doctor_id, room),
   FOREIGN KEY (room) REFERENCES room_specialization(room)
);
```

Interview Question:

- Q: What's the difference between 3NF and BCNF?
- A: 3NF allows a transitive-like dependency to persist if the determining attribute is part of some candidate key (a "prime attribute" exception). BCNF removes this exception — every determinant of a functional dependency must be a super key, with no exceptions, making it a stricter form of normalization.

Common Mistake:
Over-applying BCNF in every practical schema — in real-world systems, 3NF is usually "good enough" and fully satisfies business needs; BCNF violations are relatively rare edge cases (multiple overlapping candidate keys), and pushing normalization further than necessary can hurt query performance without meaningful integrity benefit.

---

25. Denormalization

Definition:
Denormalization is the deliberate, controlled introduction of redundancy into a normalized schema — done for **performance reasons**, typically in read-heavy systems like reporting/analytics.

Why it is needed:
Highly normalized schemas require many `JOIN`s to reconstruct meaningful data, which can be slow for read-heavy operations (e.g., dashboards, reports run thousands of times per day).

Example:

sql

```
-- Fully normalized: requires 3 joins to get a doctor's name on an appointment report
SELECT a.appointment_id, d.first_name, d.last_name, a.appointment_date
FROM appointments a
JOIN doctors d ON a.doctor_id = d.doctor_id;

-- Denormalized (for a reporting table only):
CREATE TABLE appointment_report (
    appointment_id INT,
    doctor_full_name VARCHAR(100),  -- redundant, pre-joined for speed
    appointment_date DATE
);
```

Best Practice:
Never denormalize your **primary transactional (OLTP)** tables (like our Health Clinic core schema) — keep those fully normalized for data integrity. Denormalization belongs in **separate reporting/analytics tables** (OLAP), populated periodically (e.g., via a scheduled job), where read speed matters more than write-time consistency.

Interview Question:

- Q: When would you deliberately denormalize a database, and what's the trade-off?
- A: Denormalization is used in read-heavy reporting/analytics systems to avoid expensive joins on every query, trading some storage redundancy and potential update complexity for significantly faster read performance. It should be applied selectively (e.g., separate reporting tables), never to the core transactional schema, to avoid reintroducing update/insertion/deletion anomalies.

Practice: Complete ER Diagram, Normalized Schema & Indexes for Health Clinic

sql

```
-- ============================================
-- Day 2 Practice: Fully Normalized Health Clinic Schema
-- ============================================

CREATE DATABASE IF NOT EXISTS health_clinic_db;
USE health_clinic_db;

-- Drop Day 1 basic tables to rebuild normalized version
DROP TABLE IF EXISTS patients;
DROP TABLE IF EXISTS doctors;
```

```sql
-- 1. Patients (3NF compliant)
CREATE TABLE patients (
    patient_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    date_of_birth DATE,
    gender ENUM('Male', 'Female', 'Other'),
    email VARCHAR(100) UNIQUE,
    registered_on TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- 2. Patient Phones (1NF fix -- multi-valued attribute separated)
CREATE TABLE patient_phones (
    phone_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    phone_number VARCHAR(15) NOT NULL,
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id) ON DELETE CASCADE,
    INDEX idx_patient_id (patient_id)
);

-- 3. Doctors (3NF compliant -- specialization moved out, see below)
CREATE TABLE doctors (
    doctor_id INT AUTO_INCREMENT PRIMARY KEY,
    first_name VARCHAR(50) NOT NULL,
    last_name VARCHAR(50) NOT NULL,
    phone_number VARCHAR(15) UNIQUE,
    email VARCHAR(100) UNIQUE
);

-- 4. Specializations (separate entity)
CREATE TABLE specializations (
    specialization_id INT AUTO_INCREMENT PRIMARY KEY,
    name VARCHAR(100) NOT NULL UNIQUE,
    description VARCHAR(255)
);

-- 5. Doctor <-> Specialization (M:N junction table)
CREATE TABLE doctor_specializations (
    doctor_id INT,
    specialization_id INT,
    PRIMARY KEY (doctor_id, specialization_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id) ON DELETE CASCADE,
    FOREIGN KEY (specialization_id) REFERENCES specializations(specialization_id) ON DELETE
CASCADE
);

-- 6. Appointments (core transactional table)
CREATE TABLE appointments (
    appointment_id INT AUTO_INCREMENT PRIMARY KEY,
    patient_id INT NOT NULL,
    doctor_id INT NOT NULL,
    appointment_date DATETIME NOT NULL,
    status ENUM('Scheduled', 'Completed', 'Cancelled') DEFAULT 'Scheduled',
    FOREIGN KEY (patient_id) REFERENCES patients(patient_id),
    FOREIGN KEY (doctor_id) REFERENCES doctors(doctor_id),
    INDEX idx_patient_id (patient_id),
    INDEX idx_doctor_date (doctor_id, appointment_date)   -- composite index
);
```

```
-- 7. Billing (1:1 with Appointment)
CREATE TABLE billing (
    bill_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL UNIQUE,  -- UNIQUE enforces 1:1
    amount DECIMAL(10,2) NOT NULL,
    payment_status ENUM('Pending', 'Paid', 'Refunded') DEFAULT 'Pending',
    billing_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);

-- 8. Visit History (1:1 with Appointment)
CREATE TABLE visit_history (
    visit_id INT AUTO_INCREMENT PRIMARY KEY,
    appointment_id INT NOT NULL UNIQUE,  -- UNIQUE enforces 1:1
    diagnosis VARCHAR(255),
    prescription VARCHAR(255),
    visit_notes TEXT,
    FOREIGN KEY (appointment_id) REFERENCES appointments(appointment_id)
);

-- Sample Data
INSERT INTO patients (first_name, last_name, date_of_birth, gender, email)
VALUES ('Ramesh', 'Kumar', '1979-05-14', 'Male', 'ramesh@email.com');

INSERT INTO patient_phones (patient_id, phone_number) VALUES (1, '9876543210'), (1, '9998887777');

INSERT INTO doctors (first_name, last_name, phone_number, email)
VALUES ('Anjali', 'Rao', '9123456780', 'dr.rao@clinic.com');

INSERT INTO specializations (name, description)
VALUES ('Cardiology', 'Heart-related treatment'), ('Pediatrics', 'Child healthcare');

INSERT INTO doctor_specializations (doctor_id, specialization_id) VALUES (1, 1);

INSERT INTO appointments (patient_id, doctor_id, appointment_date, status)
VALUES (1, 1, '2026-08-05 10:00:00', 'Scheduled');

INSERT INTO billing (appointment_id, amount, payment_status)
VALUES (1, 1500.00, 'Pending');

INSERT INTO visit_history (appointment_id, diagnosis, prescription, visit_notes)
VALUES (1, 'Routine Checkup', 'None', 'Patient in good health');

-- Verify with a join
SELECT p.first_name, p.last_name, d.first_name AS doctor_name,
       a.appointment_date, b.amount, v.diagnosis
FROM appointments a
JOIN patients p ON a.patient_id = p.patient_id
JOIN doctors d ON a.doctor_id = d.doctor_id
JOIN billing b ON a.appointment_id = b.appointment_id
JOIN visit_history v ON a.appointment_id = v.appointment_id;
```

Sample Output:

```
+------------+-----------+-------------+---------------------+---------+------------------+
| first_name | last_name | doctor_name | appointment_date    | amount  | diagnosis        |
+------------+-----------+-------------+---------------------+---------+------------------+
| Ramesh     | Kumar     | Anjali      | 2026-08-05 10:00:00 | 1500.00 | Routine Checkup  |
+------------+-----------+-------------+---------------------+---------+------------------+
```

Day 2 Summary

- Learned formal **ER Diagram** components: Entity, Attribute, Relationship, Cardinality, Participation, Primary/Foreign Keys
- Designed the **complete ER Diagram** for the Health Clinic Application, resolving the Doctor↔Specialization M:N relationship via a junction table
- Deep-dived into **Indexing**: B+ Tree internals, Clustered vs Non-Clustered, Composite (leftmost prefix rule), Unique, and Covering indexes
- Used `EXPLAIN` to measure real performance impact of indexes
- Covered **Normalization** from 1NF → BCNF, fixing update/insertion/deletion anomalies via functional dependency analysis
- Learned **Denormalization** as a deliberate, targeted performance trade-off for OLAP/reporting systems
- Built the **fully normalized Health Clinic schema** with proper indexes — ready for Day 3 (Joins, Stored Procedures, Triggers)

Practice Questions

1. Draw (in ASCII or on paper) the ER diagram for a "Library Management System" with Books, Members, and Borrowing records — identify cardinality and participation.
2. What's the difference between a clustered and non-clustered index? Why can a table have only one clustered index?
3. Given a composite index on `(city, age)`, will a query filtering only on `age` use this index? Why or why not?
4. Take this unnormalized table and normalize it to 3NF: `orders(order_id, customer_name, customer_email, product_name, product_price, quantity)`.
5. What's the difference between 3NF and BCNF? Give an example where a table satisfies 3NF but not BCNF.
6. When would you deliberately denormalize a database? What's the risk?

Assignment

1. Extend the Health Clinic schema: add a `rooms` table and a `doctor_room` relationship reflecting doctors assigned to specific consultation rooms.
2. Write and run `EXPLAIN` on at least 3 different queries against the `appointments` table — one with no index, one using a single-column index, one using the composite index — and note the differences in the `type` and `rows` columns.
3. Take the `patient_phones` design and verify it satisfies 1NF, 2NF, and 3NF — write a short justification for each.
4. Create a covering index for a query that reports `doctor_id, appointment_date, status` from the `appointments` table, and verify with `EXPLAIN` that `Extra` shows `Using index`.

Interview Questions (Day 2 Recap)

1. What is the difference between a strong entity and a weak entity?
2. How is a Many-to-Many relationship implemented in a relational database?
3. What's the difference between total and partial participation, and how does it affect schema design?
4. Why does InnoDB use a B+ Tree instead of a hash table for indexing?
5. What is a covering index, and how does it improve performance?
6. What is a functional dependency, and how does it relate to normalization?
7. Explain 1NF, 2NF, and 3NF with one-line definitions each.
8. When and why would you choose to denormalize a database table?

Final Interview Questions with Detailed Explanations

— ER Diagrams, Indexing, and Normalization — each with a full explanation.

Q1. What is the difference between a strong entity and a weak entity?

Explanation:
A strong entity has its own primary key and can be uniquely identified independently of any other entity — e.g., `Patient` or `Doctor` in our Health Clinic schema. A weak entity cannot be uniquely identified by its own attributes alone; it depends on a related "owner" strong entity, typically using a partial key combined with the owner's primary key to form a composite identifier. In ER diagram notation, weak entities are drawn with double-lined rectangles. Interviewers use this to check if you understand that not all data naturally has an independent identity — some data only makes sense in the context of a parent record.

Q2. Explain the different types of attributes with examples.

Explanation:
Simple attributes cannot be broken down further (e.g., `age`). Composite attributes can be split into sub-parts (e.g., `name` → `first_name` + `last_name`). Single-valued attributes hold exactly one value per entity (e.g., `date_of_birth`), while multi-valued attributes can hold several (e.g., a patient having multiple phone numbers). Derived attributes are calculated from other stored attributes (e.g., `age` derived from `date_of_birth`) and should generally not be physically stored, since they go stale. Key attributes uniquely identify an entity instance (e.g., `patient_id`). Recognizing these types during schema design directly determines whether an attribute becomes its own column, a separate table, or a computed value at query time.

Q3. How is a Many-to-Many (M:N) relationship implemented in a relational database, and why can't it use a simple foreign key?

Explanation:
A foreign key column can only point to one row on the referenced ("one") side of a relationship — it's fundamentally designed to model a 1:N relationship. In an M:N relationship, both sides can legitimately relate to multiple rows on the other side (e.g., one doctor has many specializations, and one specialization applies to many doctors). To model this correctly, you introduce a **junction/bridge table** containing foreign keys to both parent tables, with a composite primary key formed from both — as we did with `doctor_specializations`. This decomposes the single M:N relationship into two clean 1:N relationships.

Q4. What's the difference between total and partial participation in an ER diagram, and how does it affect schema design?

Explanation:
Total (mandatory) participation means every instance of an entity *must* participate in the relationship — for example, every `Appointment` must generate a `Billing` record. Partial (optional) participation means participation is not required — for example, a `Patient` can exist in the system without having booked any `Appointment` yet. This distinction directly affects schema design: total participation usually implies a `NOT NULL` foreign key (or application/trigger-level enforcement that a related row must exist), while partial participation allows the foreign key column to be `NULL` or simply have zero related rows.

Q5. Why does InnoDB use a B+ Tree instead of a hash table for indexing?

Explanation:
Hash tables provide O(1) average lookup time but only for exact-match queries, because hashing scrambles the natural order of values — you cannot efficiently perform range queries (`<`, `>`, `BETWEEN`) or sorted retrieval (`ORDER BY`) on a hash structure. A B+ Tree keeps data sorted at all times and links its leaf nodes together, enabling both fast O(log n) exact lookups and efficient sequential range scans by traversing linked leaves instead of re-searching the tree for each row. This makes B+ Trees far more versatile for the mixed query patterns (exact match, ranges, sorting) that real applications actually need.

Q6. What is the difference between a clustered index and a non-clustered (secondary) index?

Explanation:
A clustered index determines the actual **physical storage order** of table rows — the row data itself lives in the leaf nodes of this index. In InnoDB, the Primary Key is automatically the clustered index, and a table can have only one, since data can only be physically sorted one way at a time. A non-clustered (secondary) index stores the indexed column's value plus a

pointer back to the primary key, meaning a query using only a secondary index often needs a second lookup into the clustered index to retrieve the full row — unless the query is satisfied entirely by a covering index.

Q7. Given a composite index on `(doctor_id, appointment_date)`, will a query filtering only on `appointment_date` use this index? Why or why not?

Explanation:
No — this is the **leftmost prefix rule**. A composite index is physically sorted first by the leftmost column, then by subsequent columns within each value of the leftmost column. Since the index is sorted primarily by `doctor_id`, searching by `appointment_date` alone cannot leverage that sorted structure — it's like trying to find a name in a phone book sorted by last name, using only a first name. To support that query efficiently, a separate index on `appointment_date` alone (or with it as the leftmost column) would be required.

Q8. What is a covering index, and how does it improve performance?

Explanation:
A covering index is one where all the columns a query needs — in its `SELECT`, `WHERE`, and `ORDER BY` clauses — are present within the index itself. This means the database engine can answer the entire query using only the index structure, without performing the extra lookup into the clustered index (actual table data) that a normal secondary index would require. This is especially valuable for frequently-run reporting queries that consistently select the same small set of columns, since it eliminates a whole layer of disk/memory access per row.

Q9. Why wouldn't you index every column in a table?

Explanation:
While indexes dramatically speed up `SELECT`/`WHERE`/`JOIN` operations, every index must also be updated whenever the underlying data changes — so each additional index adds overhead to every `INSERT`, `UPDATE`, and `DELETE`. Additionally, indexing low-cardinality columns (columns with very few distinct values, like `gender`) rarely helps, since a large fraction of rows match any given value anyway, and the query optimizer often ignores such indexes in favor of a full table scan. The right approach is to index only columns frequently used in filtering, joining, or sorting — not blindly index everything.

Q10. What is a functional dependency, and why is it foundational to normalization?

Explanation:
A functional dependency `A → B` means that for every value of A, there is exactly one corresponding value of B — A uniquely determines B. For example, `patient_id →`

`patient_name` means knowing the patient_id tells you exactly one name. Normalization's normal forms (2NF, 3NF, BCNF) are formally defined entirely in terms of functional dependencies — specifically, eliminating "bad" dependencies (partial and transitive) that cause update, insertion, and deletion anomalies. Without understanding functional dependency, you can't rigorously justify *why* a table is or isn't in a given normal form — you'd just be pattern-matching examples.

Q11. Explain 1NF, 2NF, and 3NF with a one-line definition and example violation for each.

Explanation:
1NF requires atomic column values and no repeating groups — violated by storing comma-separated phone numbers in one column instead of a separate `patient_phones` table. **2NF** requires every non-key attribute to depend on the *entire* composite primary key, not just part of it — violated when, say, `appointment_date` depends only on `appointment_id` in a table keyed by `(appointment_id, doctor_id)`, rather than on the full composite key. **3NF** requires no non-key attribute to depend on another non-key attribute (no transitive dependency) — violated when `doctor_specialization` depends on `doctor_id`, which itself is just a non-key attribute in an `appointments` table keyed by `appointment_id`, rather than depending directly on the primary key.

Q12. What's the difference between 3NF and BCNF? Give a scenario where a table satisfies 3NF but violates BCNF.

Explanation:
3NF permits a narrow exception: a transitive-like dependency `X → Y` is tolerated if `Y` is a "prime attribute" (part of some candidate key). BCNF removes this exception entirely — every determinant of any functional dependency must be a super key, with no exceptions. The classic example is a table with overlapping candidate keys, like `(doctor_id, specialization) → room` and `room → specialization`: since `specialization` is part of a candidate key, 3NF tolerates the dependency, but BCNF does not, because `room` alone is not a super key. In practice, BCNF violations are rare edge cases, and most production schemas stop comfortably at 3NF.

Q13. What are the three main anomalies caused by an unnormalized schema, and how does normalization fix each?

Explanation:
The **update anomaly** occurs when duplicated data (e.g., a patient's phone number repeated across many appointment rows) requires updating multiple rows consistently — miss one, and data becomes contradictory. The **insertion anomaly** occurs when you cannot add certain data (e.g., a new doctor) without unrelated data also being present (e.g., at least one appointment). The **deletion anomaly** occurs when deleting one record unintentionally destroys unrelated information (e.g., deleting a patient's only appointment also erases the only

record of a doctor's specialization). Normalization fixes all three by decomposing data into smaller tables where each fact is stored exactly once, connected via foreign keys.

Q14. When would you deliberately denormalize a database, and what's the trade-off?

Explanation:
Denormalization is used when read performance matters more than write-time consistency — typically in reporting/analytics (OLAP) systems where the same complex, multi-join queries run repeatedly and joining normalized tables on every request becomes a performance bottleneck. By pre-joining or duplicating some data into a flatter structure, reads become much faster at the cost of extra storage and more complex update logic (since redundant copies must now be kept in sync). The key trade-off rule: never denormalize your core transactional (OLTP) schema — keep that fully normalized for data integrity — and instead apply denormalization only to separate, periodically-refreshed reporting tables.

Q15. In our Health Clinic schema, why is `doctor_specializations` a separate junction table instead of adding a `specialization` column directly to `doctors`?

Explanation:
Because the relationship between `Doctor` and `Specialization` is Many-to-Many — a doctor can practice multiple specializations, and a specialization is shared across many doctors. Adding a single `specialization` column to `doctors` could only model a 1:N relationship (one doctor, one specialization) and would force ugly workarounds like comma-separated values (violating 1NF) if a doctor had more than one. The `doctor_specializations` junction table, with a composite primary key of `(doctor_id, specialization_id)`, correctly decomposes the M:N relationship into two clean 1:N relationships while keeping every value atomic.

Q16. Why is `appointment_id` marked as `UNIQUE` in both the `billing` and `visit_history` tables?

Explanation:
Both `Billing` and `VisitHistory` have a 1:1 relationship with `Appointment` — each appointment should produce exactly one bill and exactly one visit record. A plain foreign key alone only enforces that `appointment_id` *exists* in the parent table; it does **not** prevent multiple billing rows from referencing the same appointment. Adding a `UNIQUE` constraint on the foreign key column enforces that each `appointment_id` can appear at most once in `billing` (and separately, at most once in `visit_history`), correctly modeling the 1:1 cardinality at the database level rather than relying on application code to prevent duplicates.
